

ORCHESTRATING AI APPLICATIONS

How do isolated LLM calls scale into real applications?

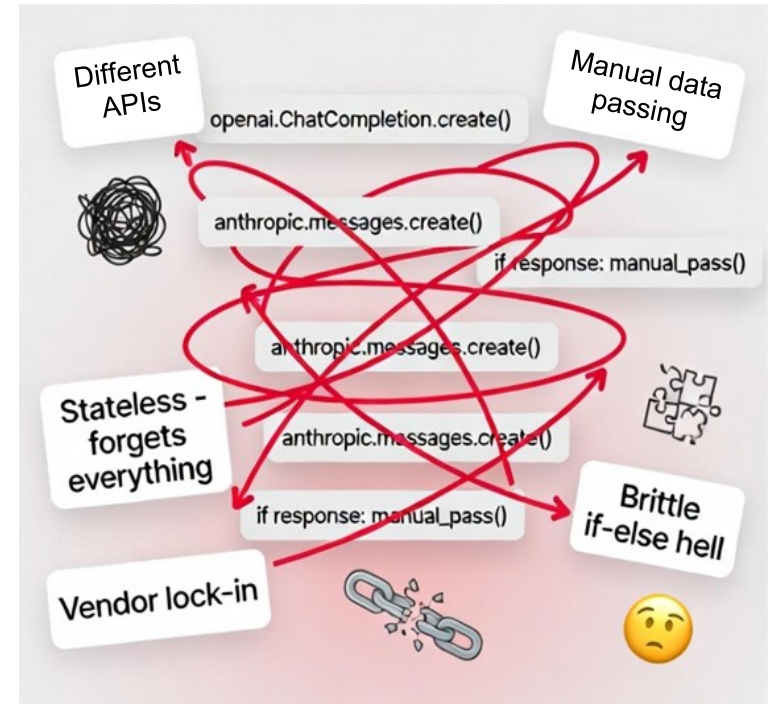
- LLMs understand language and generate fluent responses, but building real applications with them is surprisingly hard
- Modern LLM systems are not single prompts
- They involve:
 - Multiple LLM calls
 - External tools (search, DBs, APIs)
 - Conditional logic (if-else based on outputs)
 - Intermediate representations

Orchestration is the coordination of multiple steps, context, external tools, and memory across massive, stateless models

THE ORCHESTRATION PROBLEM

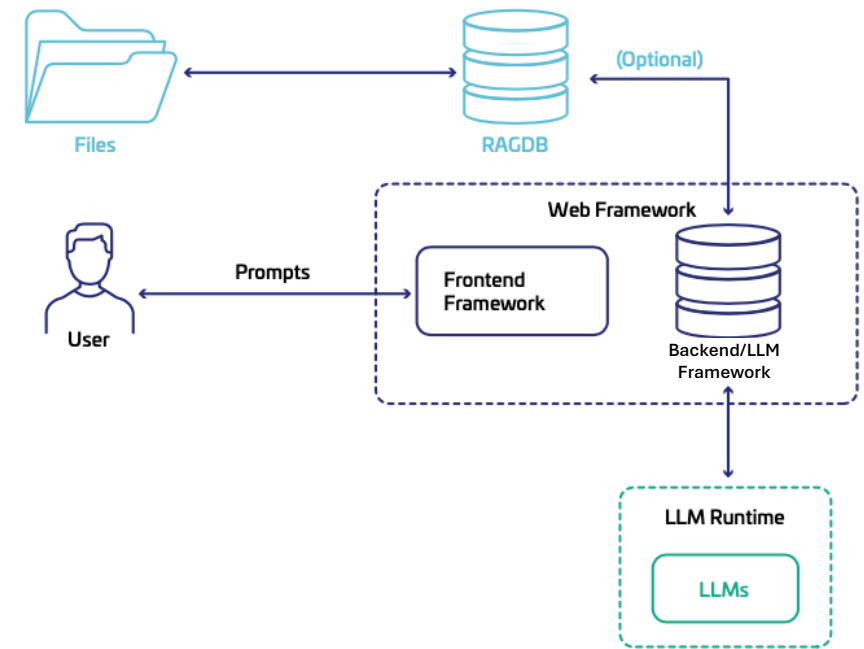
Raw LLM code becomes messy very quickly

- Different providers = different code styles (vendor lock-in)
- Multiple prompts scattered across the project files
- No built-in chaining → manual, messy data passing, and hidden dependencies between steps
- Manual parsing of outputs
- Statelessness → the model forgets previous messages immediately
- Scaling to multi-step reasoning requires brittle "spaghetti code"



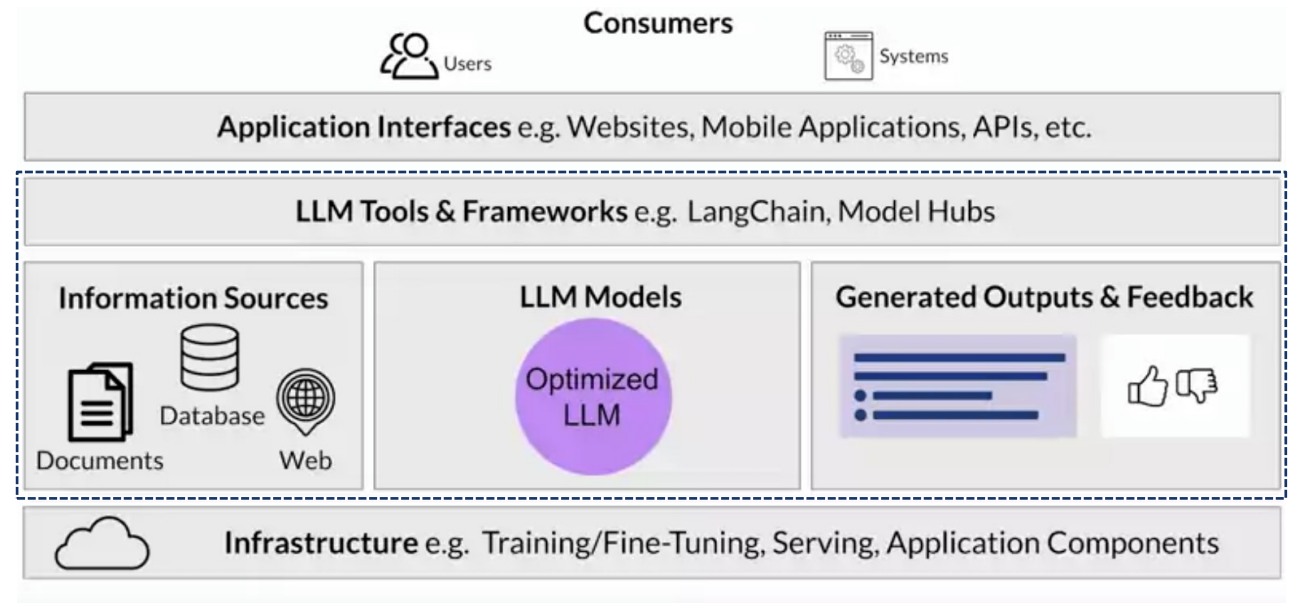
WHY FRAMEWORKS EXIST?

- As applications grow, the system must handle more than a single model call
- We need to manage multiple prompts, repeated model invocations, structured inputs and outputs
- Frameworks exist to organise complexity
- A framework is a structured abstraction over common patterns
- It provides standard building blocks and clear interfaces between components that extend functionalities in predictable and defined ways
- LLM frameworks help to manage prompt construction, data flow within steps, model configurations and output handling



LANGCHAIN

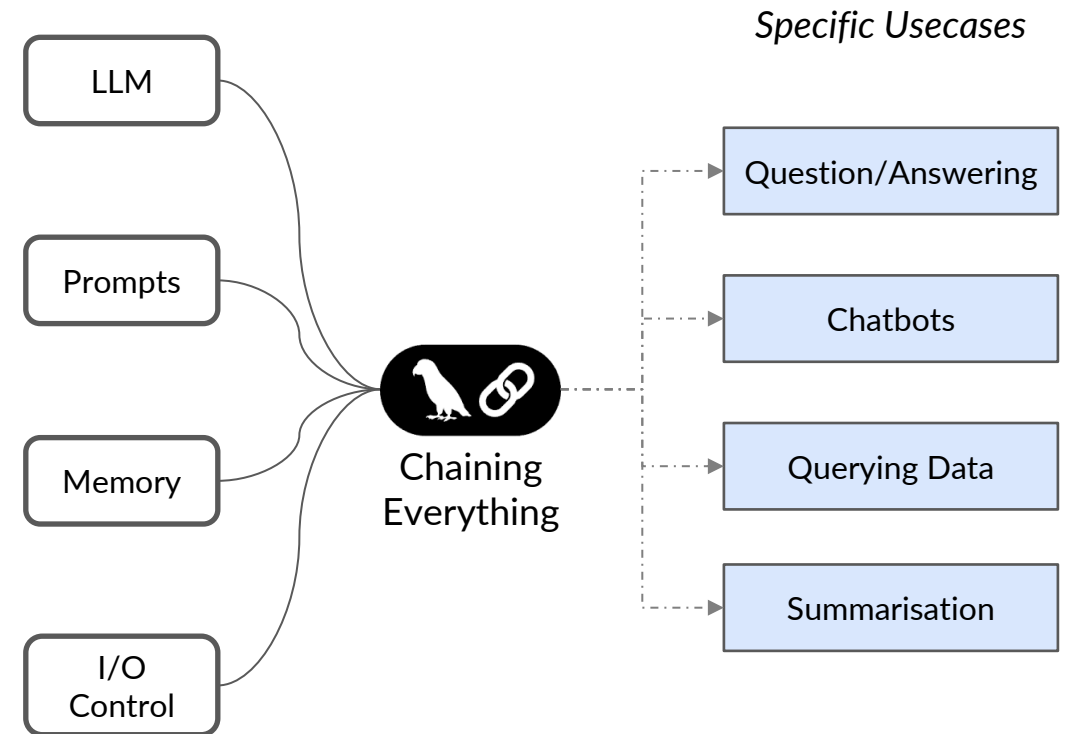
- LangChain is a modular ecosystem for building LLM-based applications
- It is an orchestration framework for LLM usage
- It focuses on:
 - Organising prompts
 - Managing model calls
 - Structuring multi-step workflows
 - Handling inputs and outputs cleanly
- It sits on top of LLM APIs, not against them
- It doesn't train model or change its behaviour; it is used to organise the model usage



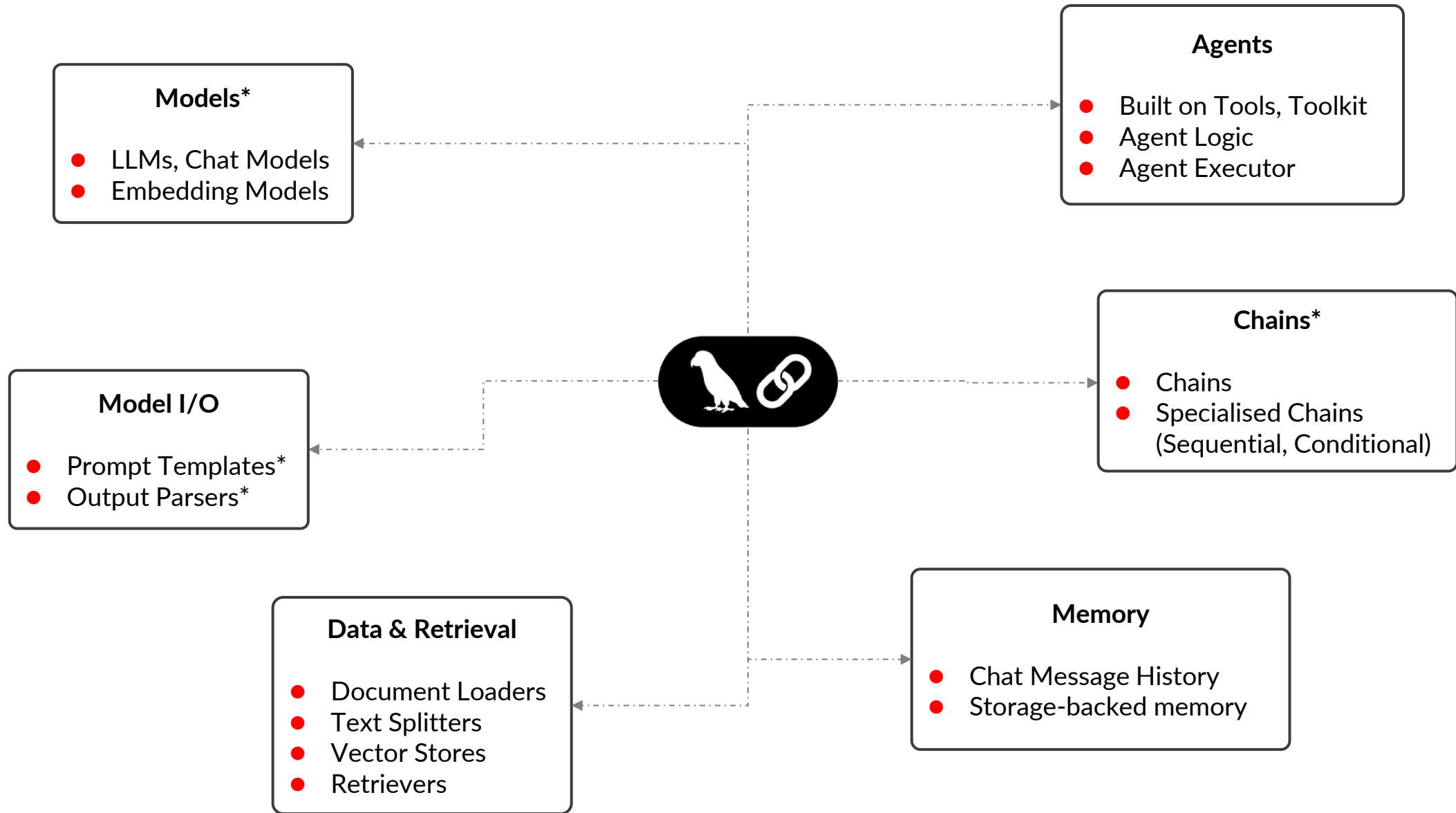
WHAT LANGCHAIN IMPROVES

○ LangChain helps with:

- **Models:** Provides model wrappers and interfaces to directly use LLM APIs
- **Prompts:** Dynamic templates replacing hard-coded strings
- **Model I/O:** Clear input-output contracts
- **Memory:** Built-in conversation history solving the stateless problem
- **Chaining:** Automatic step-by-step pipelines (no manual passing)



COMPONENTS OF LANGCHAIN



* : Core Primitives in LC

MODELS AND INPUT-OUTPUT

Component 1: Models

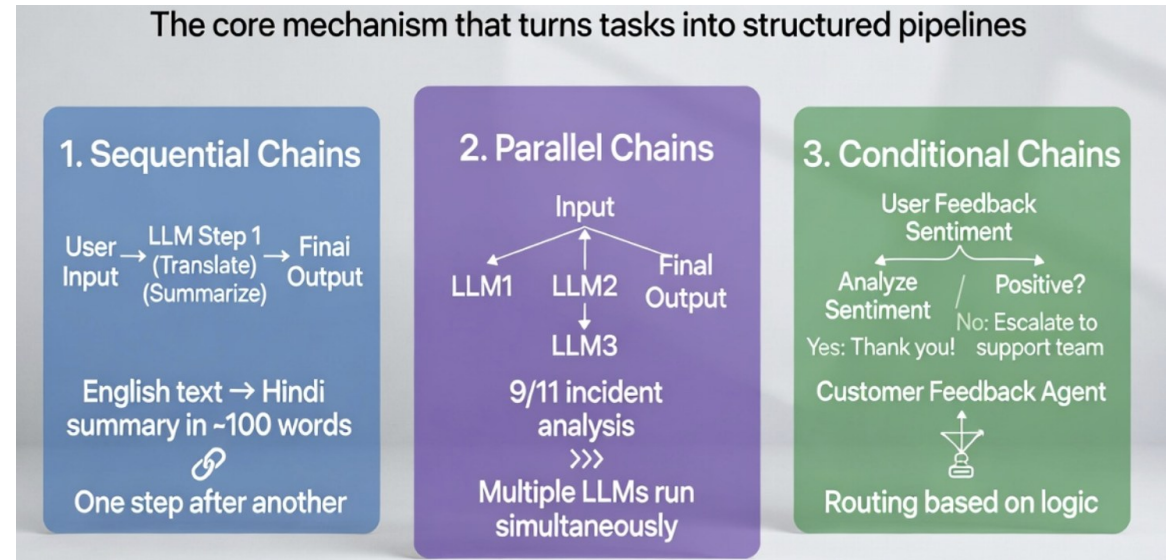
- The core interfaces of the framework
- Language Models: Text-in → Text-out (OpenAI, Anthropic, Gemini)
- Embedding Models: Text-in → Vector-out (Essential for semantic search)

Component 2: Input-Output

- LLMs are highly sensitive to input phrasing
- LangChain offers **Prompt Templates**, which offer:
 - Dynamic Prompts: Reusable templates with *{placeholders}*
 - Role-Based Prompts: Defining "System" vs "User" roles
 - Few-Shot Prompts: Injecting examples to control formatting
- **Output parsers** convert unstructured text from language models into structured, actionable data formats such as JSON, lists, or Python objects

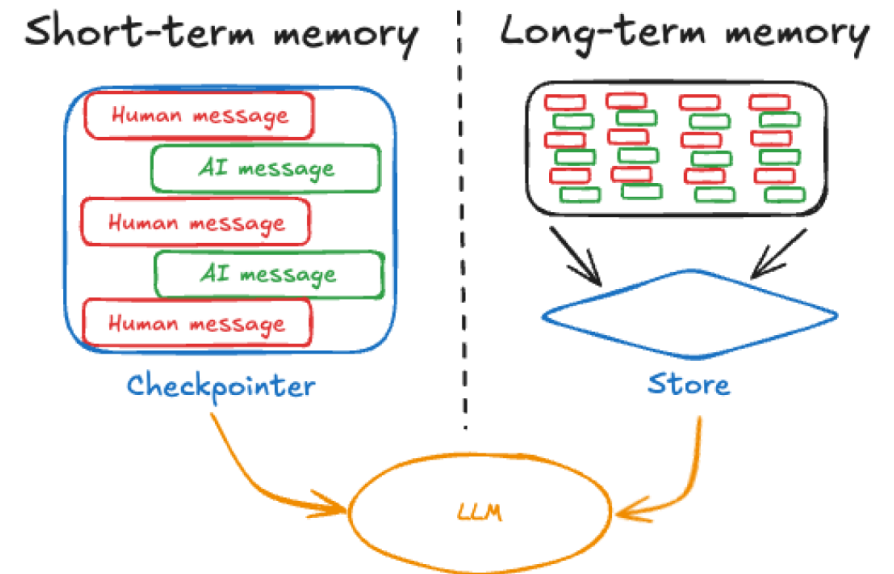
CHAINS

- Chains are the core mechanism that turns tasks into structured pipelines
- Sequential Chains: *output of step 1 → step 2 input*
- Parallel Chains: Running multiple LLMs simultaneously
- Conditional Chains: Routing based on logic (e.g., *if sentiment = bad → support path*)



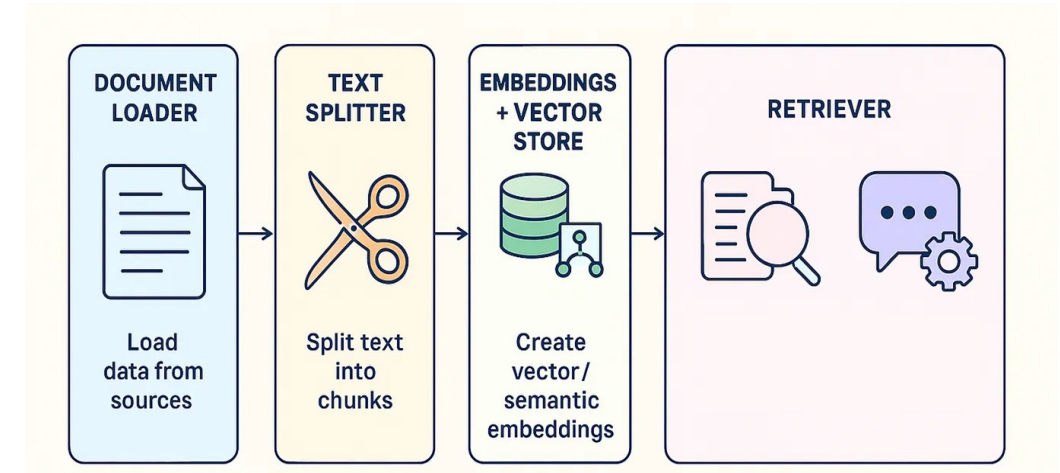
MEMORY

- Large Language Models are inherently stateless
- Memory mechanisms provide continuity by maintaining context across interactions
- Short-term memory maintains recent conversational context within the prompt, using mechanisms such as full buffers, sliding windows, or summaries to manage token usage
- Long-term memory stores information externally, allowing context to persist across sessions and be retrieved when needed



DATA & RETRIEVAL

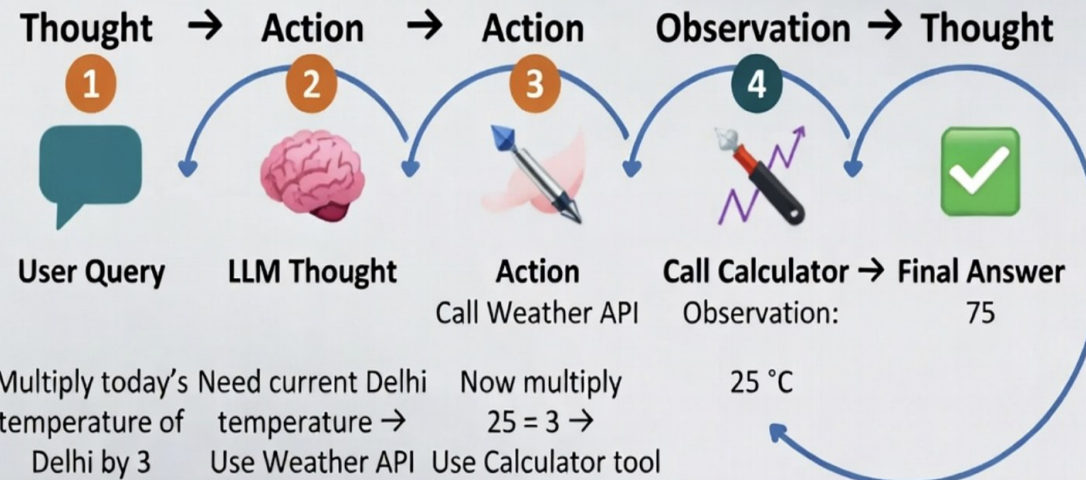
- Connectors and loaders help in connecting the LLM to private/external knowledge
- The 4-Part Flow:
 1. Document Loaders (PDFs, CSVs, etc.)
 2. Text Splitters
 3. Vector Stores
 4. Retrievers



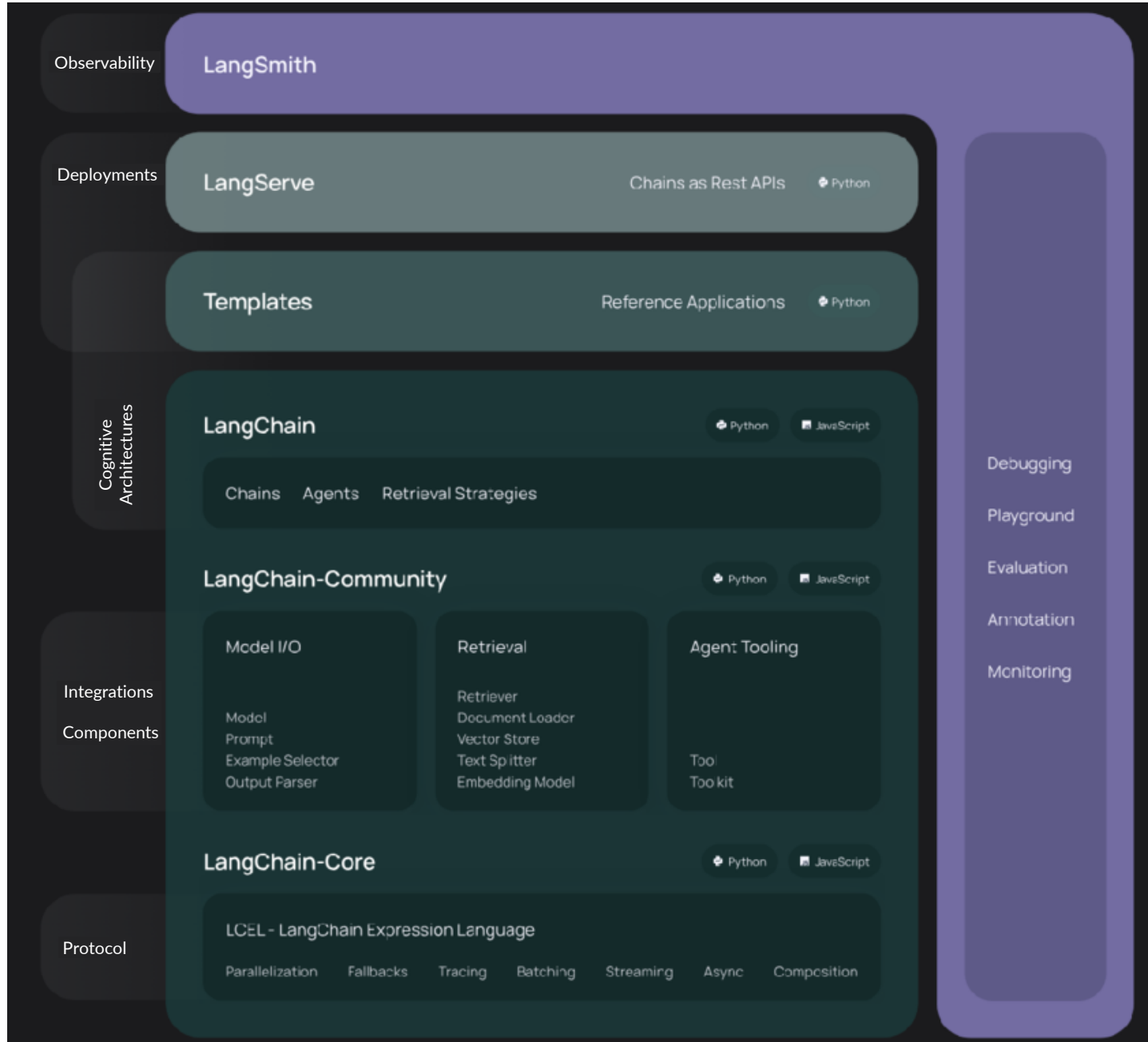
AGENTS

- A chatbot focuses on understanding input and generating responses
- An AI agent goes further by reasoning about the task and taking actions
- In LangChain, agents combine an LLM, tool access (APIs, calculators, databases), and an execution loop that selects and performs actions

Chatbot replies → Agent reasons + acts using tools



THE LANGCHAIN ECOSYSTEM



LangChain shifts AI systems from one-off API calls to software engineering